

Informe – Trabajo Práctico Integrador (TPI)

Gestión de Datos de Países en Python

Institución: Universidad Tecnológica Nacional (UTN)

Carrera: Tecnicatura Universitaria en Programación

Materia: Programación I

Trabajo: Trabajo Práctico Integrador

Título: Gestión de Datos de Países en Python

Integrantes:

- Brest Sofía
- Nieva Agustina

Índice

1. Introducción
2. Objetivos
3. Marco teórico
4. Arquitectura del sistema
5. Diseño del sistema
6. Funcionalidades implementadas
7. Implementación técnica
8. Pruebas del sistema
9. Dificultades encontradas
10. Conclusiones
11. Bibliografía

1. Introducción

El presente informe documenta el desarrollo de una aplicación en Python destinada a la gestión de datos de países utilizando estructuras de datos, modularización y persistencia mediante archivos CSV.

El sistema permite realizar operaciones de alta, modificación, búsqueda, filtrado, ordenamiento y análisis estadístico, integrando los contenidos fundamentales de la materia Programación I.

El proyecto se desarrolló siguiendo buenas prácticas de programación estructurada, separación de responsabilidades y control de versiones mediante Git y GitHub.

El resultado es un sistema funcional, escalable y de fácil mantenimiento, adecuado para el manejo de información tabular.

2. Objetivos

Los objetivos del trabajo se dividen en dos categorías: **conceptuales** y **técnicos**.

Objetivos conceptuales

- Comprender y aplicar estructuras de datos fundamentales (listas y diccionarios).
- Implementar modularización para mejorar la organización del código.
- Aplicar técnicas de filtrado, ordenamiento y búsqueda.
- Comprender el funcionamiento de archivos CSV como medio de persistencia.

Objetivos técnicos

- Desarrollar un sistema interactivo basado en consola.
- Implementar un menú principal que permita acceder a todas las funcionalidades.
- Garantizar validaciones robustas para evitar errores del usuario.
- Utilizar Git y GitHub para control de versiones y trabajo colaborativo.
- Documentar el proyecto de manera profesional.

3. Marco teórico

A continuación se describen los conceptos teóricos aplicados en el desarrollo del sistema.

Listas

Estructura de datos ordenada y mutable que permite almacenar múltiples elementos.

En este proyecto, cada elemento de la lista representa un país.

Diccionarios

Estructura clave-valor que permite representar entidades complejas.

Cada país se modela como un diccionario con claves:

- nombre
- población
- superficie
- continente

Modularización

Consiste en dividir el programa en módulos independientes, cada uno con una responsabilidad específica.

Esto mejora la legibilidad, el mantenimiento y la escalabilidad del sistema.

Archivos CSV

Formato de texto plano utilizado para almacenar datos tabulares.

Python permite su lectura y escritura mediante el módulo csv, lo que facilita la persistencia de datos.

Algoritmos aplicados

- **Búsqueda lineal:** para localizar países por nombre.
- **Filtrado condicional:** para seleccionar países según criterios específicos.
- **Ordenamiento:** utilizando claves como nombre, población o superficie.
- **Estadísticas básicas:** cálculo de máximos, mínimos, promedios y agrupamientos.

4. Arquitectura del sistema

El sistema se organiza en tres capas claramente diferenciadas:

1. Capa de presentación

- Archivo main.py
- Contiene el menú principal y coordina la interacción con el usuario.

2. Capa lógica

Carpeta funciones/ con módulos especializados:

- altas.py
- actualizar.py

- `busquedas.py`
- `filtros.py`
- `ordenamiento.py`
- `estadisticas.py`
- `datos.py`

Cada módulo implementa una funcionalidad específica del sistema.

3. Capa de datos

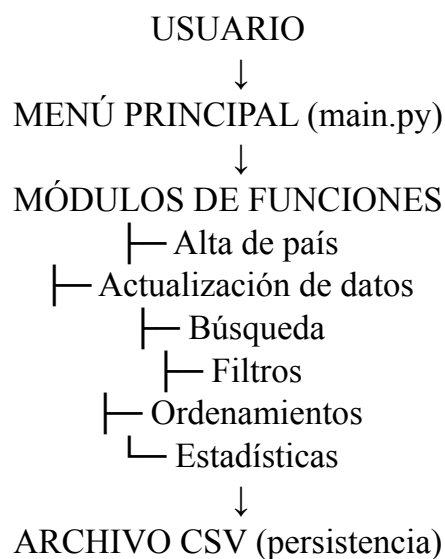
- Archivo `países.csv`
- Contiene la información persistente del sistema.

Esta arquitectura permite una separación clara entre interfaz, lógica y datos.

5. Diseño del sistema

El diseño del sistema se basa en un flujo de interacción simple y directo:

Código



El menú principal actúa como punto de entrada y distribuye las operaciones hacia los módulos correspondientes.

6. Funcionalidades implementadas

El sistema incluye las siguientes funcionalidades:

- **Alta de países:** permite registrar nuevos países con todos sus datos.

- **Actualización:** modifica población y superficie de un país existente.
- **Búsqueda:** permite localizar países por coincidencia exacta o parcial.
- **Filtros:** por continente, rango de población y rango de superficie.
- **Ordenamientos:** por nombre, población o superficie.
- **Estadísticas:**
 - país con mayor y menor población
 - promedio de población
 - promedio de superficie
 - cantidad de países por continente
- **Persistencia:** guardado automático en el archivo CSV.

7. Implementación técnica

El proyecto utiliza:

Estructuras de datos

- Lista de diccionarios para almacenar los países.

Modularización

Cada módulo implementa una responsabilidad única, por ejemplo:

- `altas.py`: registro de países
- `actualizar.py`: modificación de datos
- `busquedas.py`: búsqueda exacta y parcial
- `filtros.py`: filtros por criterios
- `ordenamiento.py`: ordenamientos ascendentes y descendentes
- `estadisticas.py`: cálculos estadísticos
- `datos.py`: lectura y escritura del CSV

Persistencia

El archivo `países.csv` se carga al iniciar el programa y se actualiza al finalizar.

Control de versiones

Se utilizó Git para registrar cambios y GitHub como repositorio remoto.

8. Pruebas del sistema

Se realizaron pruebas manuales para validar cada funcionalidad.

Caso 1: Alta de país

- Entrada: “Argentina”
- Resultado: país agregado correctamente.

Caso 2: Búsqueda parcial

- Entrada: “arg”
- Resultado: se encuentra “Argentina”.

Caso 3: Filtro por población

- Entrada: rango 50M–100M
- Resultado: lista filtrada correctamente.

Caso 4: Estadísticas

- Resultado: valores correctos para máximos, mínimos y promedios.

Caso 5: Ordenamiento

- Entrada: ordenar por superficie descendente
- Resultado: lista ordenada correctamente.

9. Dificultades encontradas

Durante el desarrollo se presentaron las siguientes dificultades:

- Resolución de conflictos en Git durante el trabajo colaborativo.
- Validación de entradas del usuario para evitar errores de formato.
- Organización modular del proyecto y distribución adecuada de funciones.
- Manejo correcto del archivo CSV, especialmente en casos de datos incompletos o mal formateados.
- Coordinación entre módulos para mantener coherencia en el flujo del sistema.

10. Conclusiones

El desarrollo del TPI permitió integrar los contenidos fundamentales de Programación I, aplicando estructuras de datos, modularización, manejo de archivos y control de versiones.

El sistema final es **funcional, robusto, modular y escalable**, cumpliendo con los objetivos planteados y demostrando un dominio adecuado de las herramientas utilizadas.

Además, el trabajo colaborativo mediante Git y GitHub permitió adquirir experiencia en metodologías de desarrollo reales, fortaleciendo habilidades técnicas y organizativas.

11. Bibliografía

- Documentación oficial de Python: <https://docs.python.org/3/>
- Material de cátedra UTN
- Documentación del módulo CSV en Python
- Recursos de Git y GitHub